SHERLOCK

Security Review For OpenSouk



Collaborative Audit Prepared For: **OpenSouk**
Lead Security Expert(s): **eeyore**
Kvar
Date Audited: **September 21 - September 26, 2026**

Introduction

OpenSouk is the first referral network built for AI agents. OpenSouk Contracts v1 is the first release of the protocol.

It covers the following:

- Atomic four-way payment split on purchase – immediate reviewer commission, held-back commission, platform fee and merchant net – across two rails: x402 and MPP charge.
- Escrowed settlement of the held-back commission and of buyer cashback with a permissionless fallback after a timeout.
- On-chain review anchoring via two paths: purchases through OpenSouk, with escrow records proving the purchase, and purchases outside OpenSouk, with EIP-712 oracle-signed verifications.
- Merchant and product registries with per-merchant RBAC
- A time-locked address registry as the single source of truth for protocol addresses
- Agent identity supplied by an external ERC-8004 ERC-721 registry

An exhaustive explanation of the system can be found in the contracts README

Scope

Repository: OpenSoukAI/contracts-audit

Audited Commit: 6992f8beea5d37f347299b67ea18a5c942571eb2

Final Commit: 3fe93bdcf09361a94312c8c3f349828f7a05c84b

Files:

- src/CommissionEscrow.sol
- src/libraries/ProtocolConstants.sol
- src/libraries/SplitMath.sol
- src/MerchantAccessControl.sol
- src/MerchantRegistry.sol
- src/MerchantRoles.sol
- src/ProductRegistry.sol
- src/ProtocolAddressRegistry.sol
- src/ReviewRegistry.sol
- src/SplitRouter.sol

Final Commit Hash

3fe93bdcf09361a94312c8c3f349828f7a05c84b

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	2	12

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Pausing `CommissionEscrow` disables user and oracle actions but does not pause escrow deadlines, potentially causing users to lose payout opportunities [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/12

Summary

Pausing `CommissionEscrow` disables user and oracle actions but does not pause escrow deadlines, potentially causing users to lose payout opportunities.

Vulnerability Detail

When `CommissionEscrow` is paused, `bind()` and all settlement functions are disabled. However, `windowEnd` and the additional 10 day fallback period continue running.

Consider this scenario:

1. `CommissionEscrow` is paused shortly before an escrow's `windowEnd`.
2. The buyer cannot call `bind()` or receive cashback, and the oracle cannot settle the escrow.
3. The pause continues beyond `windowEnd + 10 days`.
4. The contract is unpaused.
5. Anyone immediately calls `triggerSettleFallback()`.
6. The held-back commission is sent to the treasury, while the cashback is returned to the reviewer.

The pause is intended to stop activity during an emergency, but it unintentionally consumes the time during which users and the oracle were expected to act.

Impact

A buyer may lose their cashback despite being unable to bind or receive it during the pause. Similarly, a reviewer may lose their potential held-back commission because fallback settlement becomes immediately available upon unpausing.

Code Snippet

```
function bind(uint256 escrowId, uint256 agentId)
    external
    nonReentrant
    whenNotPaused
{
    // ...
}

function settleHeldBack(uint256 escrowId, uint8 unlockPct)
    external
    onlyRole(ORACLE_ROLE)
    nonReentrant
    whenNotPaused
{
    // ...
}

function releaseCashback(uint256 escrowId)
    external
    onlyRole(ORACLE_ROLE)
    nonReentrant
    whenNotPaused
{
    // ...
}

function triggerSettleFallback(uint256 escrowId)
    external
    nonReentrant
    whenNotPaused
{
    // ...

    uint256 eligibleAt = uint256(rec.windowEnd) + 10 days;

    if (block.timestamp < eligibleAt) {
        revert TooEarly(escrowId, eligibleAt, block.timestamp);
    }

    // ...
}
```

Tool Used

Manual Review

Recommendation

Exclude paused time from escrow deadlines by extending affected deadlines by the pause duration. Alternatively, introduce a sufficient grace period after unpausing before fallback settlement can be triggered.

Issue M-2: `recoverUSDC()` does not invalidate the refunded `paymentRef` [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/15

Summary

`recoverUSDC()` refunds a parked charge but leaves its `paymentRef` unused, so anyone can later settle the refunded attestation with another buyer's funds.

Vulnerability Detail

`splitCharge()` guards replays only with `_chargeSettled[paymentRef]`, and attestations have no deadline. When a charge reverts after delivery, the README says the admin refunds it with `recoverUSDC()`, which never marks the `paymentRef` as settled. The attestation stays valid and is already public in the calldata of the failed call.

Consider this scenario:

1. Buyer A sends 100 USDC and the oracle signs `ref_A`.
2. The product is inactive, so `splitCharge(ref_A)` reverts.
3. Admin refunds A with `recoverUSDC(A, 100e6)`.
4. The product is reactivated and buyer B sends 100 USDC.
5. Anyone calls `splitCharge(ref_A)`, which settles A's purchase with B's 100 USDC.
6. `splitCharge(ref_B)` reverts `InsufficientChargeBalance`.

Impact

A later buyer loses their full payment, because a refunded charge is settled a second time from the pooled funds.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L443-L447

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L390

Tool Used

Manual Review

Recommendation

Consider invalidating the `paymentRef` when refunding a charge:

```
-    function recoverUSDC(address to, uint256 amount) external
 ↪  onlyRole(DEFAULT_ADMIN_ROLE) {
+    function recoverUSDC(address to, uint256 amount, bytes32 paymentRef) external
 ↪  onlyRole(DEFAULT_ADMIN_ROLE) {
         if (to == address(0)) revert ZeroRecipientAddress();
+        if (paymentRef != bytes32(0)) _chargeSettled[paymentRef] = true;
         _safeTransfer(to, amount);
```

Issue L-1: A blacklisted reviewer can block the entire fallback settlement, including the treasury payout [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/11

Summary

A blacklisted reviewer can block the entire fallback settlement, including the treasury payout.

Vulnerability Detail

`CommissionEscrow::triggerSettleFallback()` sends held-back funds to the treasury and cashback to the reviewer within the same transaction. This function is intended as a safety mechanism when the oracle is unavailable.

However, If USDC blacklists the reviewer's payout address, or the reviewer transfers their NFT to a blacklisted address, the cashback transfer reverts. This rolls back the entire transaction, including the earlier treasury transfer.

The reviewer does not lose their entitlement to the funds. If they eventually want to collect them, they can change their payout address and retry the settlement successfully.

However, the admin also has a recovery path for the treasury funds. It can grant `ORACLE_ROLE` to a new address, which can call `settleHeldBack(escrowId, 0)`. This sends the full held-back amount to the treasury without attempting a transfer to the blacklisted reviewer.

Impact

The fallback mechanism becomes unusable, and both payout legs may remain stuck until the reviewer changes their payout address. However, the admin can recover the treasury leg by granting `ORACLE_ROLE` to a new address and settling the held-back amount separately.

Code Snippet

```
if (rec.state != EscrowState.Settled) {
    rec.state = EscrowState.Settled;
    heldAmt = rec.heldBackAmt;
    if (heldAmt > 0) _safeTransfer(treasury, heldAmt);
}
```

```
if (!rec.cashbackReleased) {
    rec.cashbackReleased = true;
    cashbackSent = rec.cashbackAmt;

    if (cashbackSent > 0) {
        address reviewer = _resolveWallet(rec.reviewerAgentId);
        _safeTransfer(reviewer, cashbackSent);
    }
}
```

Tool Used

Manual Review

Recommendation

Settle the held-back and cashback legs independently so that a failed transfer in one leg cannot block settlement of the other.

Discussion

lajosdeme

Thanks @Kvarr - I think this makes sense. However, instead of your recommendation to settle the two legs independently, what do you think about wrapping the cashback transfer in a `try/catch` with using `SafeERC20::trySafeTransfer()` instead? Also I think it's worth doing the same check in `CommissionEscrow::settleHeldBack()` which also has the same flaw. Do you agree?

Kvarr

> Thanks @Kvarr - I think this makes sense. However, instead of your recommendation to settle the two legs independently, what do you think about wrapping the cashback transfer in a `try/catch` with using `SafeERC20::trySafeTransfer()` instead? Also I think it's worth doing the same check in `CommissionEscrow::settleHeldBack()` which also has the same flaw. Do you agree?

Firstly, wrapping `SafeERC20::trySafeTransfer()` in `try/catch` would not catch the failure because it returns `false` instead of reverting. You would need to check the returned boolean and keep `rec.cashbackReleased` as `false` when the transfer fails, setting it to `true` only after a successful transfer.

Regarding the second question, `settleHeldBack()` does not have exactly the same issue because it can be called with `unlockPct == 0`, skipping the reviewer transfer and sending everything to the treasury. However, if you want to preserve the reviewer's share when their transfer fails, similar independent accounting can also be implemented there.

Issue L-2: `ReviewRegistry::_requireProductActive()` reverts with `ProductNotActive` when the merchant itself is inactive [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/13

Summary

`ReviewRegistry::_requireProductActive()` reverts with `ProductNotActive` when the merchant itself is inactive, which is inconsistent with `SplitRouter`.

Vulnerability Detail

`ProductRegistry::isProductActive()` returns false when either the product or its merchant is inactive. As a result, `ReviewRegistry::_requireProductActive()` always reverts with `ProductNotActive`, even when the actual reason is that the merchant is inactive or suspended.

In contrast, `SplitRouter` checks the merchant separately and reverts with the clearer `MerchantNotActive` error.

Impact

No security impact. The revert reason is simply wrong and differs from the error returned by `SplitRouter` in the same scenario.

Code Snippet

```
function _requireProductActive(uint256 merchantId, bytes32 productId) internal view
↪  {
    if (
        !IProductRegistry(
            _PROTOCOL.getAddress(_PROTOCOL.PRODUCT_REGISTRY())
        ).isProductActive(merchantId, productId)
    ) {
        revert ProductNotActive(merchantId, productId);
    }
}
```

Tool Used

Manual Review

Recommendation

Add a separate merchant check and revert with `MerchantNotActive`.

Issue L-3: `SplitRouter::route()` does not allow the configured agentWallet to be the buyer [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/14

Summary

`SplitRouter::route()` does not allow the configured agentWallet to be the buyer.

Vulnerability Detail

`SplitRouter::route()` requires the buyer to be the owner of `buyerAgentId`:

```
if (IERC721(address(_AGENT_REGISTRY)).ownerOf(a.buyerAgentId) != buyer) {
    revert InvalidBuyerAgent(a.buyerAgentId, buyer);
}
```

Other functions allow both the NFT owner and the configured `agentWallet`. Because of this, a delegated wallet has to purchase using `SENTINEL_BUYER` and bind the escrow afterward.

Impact

The configured `agentWallet` cannot purchase using its agent ID. It must use `SENTINEL_BUYER` and bind the escrow in a separate transaction.

Code Snippet

```
if (a.buyerAgentId != SENTINEL_BUYER) {
    if (IERC721(address(_AGENT_REGISTRY)).ownerOf(a.buyerAgentId) != buyer) {
        revert InvalidBuyerAgent(a.buyerAgentId, buyer);
    }
}
```

Tool Used

Manual Review

Recommendation

Allow the purchase when the buyer is either the NFT owner or the configured `agentWallet`.

Issue L-4: Referenced review and `productType` are not checked against the purchased product [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/16

Summary

Neither `route()` nor `splitCharge()` checks that the referenced `reviewId` belongs to the purchased `merchantId` and `productId`, or that `productType` matches the product in `ProductRegistry`.

Vulnerability Detail

`_settleAndDeposit()` reads only `agentId` from the referenced review and pays that agent the commission. The review's own `merchantId` and `productId` are never compared with the attribution. `productType` is copied from calldata into the escrow record without being read from `ProductRegistry`.

The README treats attribution as an off-chain assumption, verified by the facilitator on `route()` and the oracle on `splitCharge()`. The contract already holds the data, so this assumption should also be checked on-chain as defence in depth.

Impact

A wrong or stale attribution pays commission to the author of an unrelated review and stores a wrong `productType` on the escrow record.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L290-L294

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L286

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L148

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L407

Tool Used

Manual Review

Recommendation

Consider validating the review and the product type against the on-chain records:

```
         if (c.reviewId != 0) {
-            reviewerAgentId =
-            IReviewRegistry(_PROTOCOL.getAddress(_PROTOCOL.REVIEW_REGISTRY())).get
 ↪  Review(c.reviewId).agentId;
+            IReviewRegistry.Review memory r =
+                IReviewRegistry(_PROTOCOL.getAddress(_PROTOCOL.REVIEW_REGISTRY()))
 ↪  .getReview(c.reviewId);
+            if (r.merchantId != c.merchantId || r.productId != c.productId) revert
 ↪  ReviewProductMismatch();
+            reviewerAgentId = r.agentId;
         }
```

Also consider reading `productType` from `ProductRegistry.getProduct()` instead of taking it from calldata.

Issue L-5: Platform fee has no reasonable upper bound [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/17

Summary

`setPlatformFeeBps()` and the constructor accept any fee up to 100%, while the fee is taken entirely from the merchant's share.

Vulnerability Detail

The platform fee is the only rate in the split that the merchant does not control. The merchant can choose any commission up to 100%, but the fee is set by the admin and is only checked against `BPS_DENOM`. At 10000 bps the merchant receives nothing, and the router clamps commission to `10_000 - platformFeeBps`, so the fee also crowds out the reviewer commission.

Impact

Merchants have no on-chain guarantee on the maximum share the protocol can take from their sales.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L454

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L115

Tool Used

Manual Review

Recommendation

Consider capping the platform fee at a reasonable maximum, for example 20%:

```
+    uint256 private constant MAX_PLATFORM_FEE_BPS = 2_000;
-        if (newBps > BPS_DENOM) revert FeeTooHigh(newBps);
+        if (newBps > MAX_PLATFORM_FEE_BPS) revert FeeTooHigh(newBps);
```

Issue L-6: Use `ReentrancyGuardTransient` instead of the deprecated `ReentrancyGuard` [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/18

Summary

`SplitRouter` and `CommissionEscrow` inherit the storage-based `ReentrancyGuard`, which OpenZeppelin marks as deprecated in favour of `ReentrancyGuardTransient`.

Vulnerability Detail

The pinned OpenZeppelin 5.6.1 `ReentrancyGuard` states: "Deprecated. This storage-based reentrancy guard will be removed and replaced by the {ReentrancyGuardTransient} variant in v6.0." The project already compiles with `pragma^0.8.24` and `evm_version = "cancun"`, and Base supports EIP-1153, so the transient variant is a drop-in replacement.

Impact

Every `nonReentrant` call pays for persistent `SSTORE`s instead of cheaper transient storage, and the code depends on an API scheduled for removal.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L18

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/CommissionEscrow.sol#L8

Tool Used

Manual Review

Recommendation

Consider switching both contracts to `ReentrancyGuardTransient`:

```
-import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
+import {ReentrancyGuardTransient} from
↪  "@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol";
```

When moving to transient storage, also consider bumping the compiler to at least `0.8.34`, which fixes the high-severity transient storage bug `TransientStorageClearingHelperCollision` present in `0.8.28` to `0.8.33`.

Issue L-7: `AGENT_REGISTRY` can be re-pointed although it must never change [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/19

Summary

`setAddress()` accepts the `AGENT_REGISTRY` key, although the code and README state that this address is cached as an immutable by four contracts and must never be re-pointed.

Vulnerability Detail

`MerchantRegistry`, `ReviewRegistry`, `CommissionEscrow` and `SplitRouter` each read `AGENT_REGISTRY` from PAR once in their constructor and cache it as an immutable. The README (§4) and the PAR comments say it is "never re-pointed", but `setAddress()` has no check for this key. After a re-point, only newly deployed contracts use the new registry, so they silently diverge from the existing ones.

Impact

A re-point makes existing and newly deployed contracts resolve the same `agentId` to different identities.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/ProtocolAddressRegistry.sol#L117-L124

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/ProtocolAddressRegistry.sol#L38

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L118-L121

Tool Used

Manual Review

Recommendation

Consider enforcing the invariant in `setAddress()`:

```
     function setAddress(bytes32 key, address newValue) external onlyOwner {
         if (newValue == address(0)) revert ZeroAddress();
+        if (key == AGENT_REGISTRY && _addresses[key] != address(0)) revert
 ↪  ImmutableKey(key);
```

If the identity registry migration from README §4 must stay possible, consider instead documenting that `AGENT_REGISTRY` may only be re-pointed together with redeploying all four caching contracts.

Issue L-8: Constructor does not emit events for the initial fee and cashback cap [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/20

Summary

The `SplitRouter` and `ProtocolAddressRegistry` constructors set configuration values without emitting the events their setters emit.

Vulnerability Detail

`SplitRouter` sets `_platformFeeBps` and `_maxCashbackBpsOfCommission` without `PlatformFeeUpdated` and `MaxCashbackBpsUpdated`. `ProtocolAddressRegistry` sets `_timelockDelay` without `TimelockDelayUpdated`, although the same constructor already emits `AddressUpdated` for every seeded key. `Deploy.s.sol` passes a zero delay and raises it later, so the delay event appears there, but any other deployment with a non-zero delay leaves no trace.

Impact

Off-chain tooling has no event trail for the initial fee, cashback cap and timelock delay.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L122-L123

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/ProtocolAddressRegistry.sol#L100

Tool Used

Manual Review

Recommendation

Consider emitting the events in both constructors:

```
        _platformFeeBps = platformFeeBps_;
+       emit PlatformFeeUpdated(0, platformFeeBps_);
```

```
        _maxCashbackBpsOfCommission = SplitMath.MULTIPLIER_MIN;
+       emit MaxCashbackBpsUpdated(0, SplitMath.MULTIPLIER_MIN);
```

```
        _timelockDelay = timelockDelay_;
+       emit TimelockDelayUpdated(0, timelockDelay_);
```

Issue L-9: `bind()` always reads `agentWallet`, even when the caller is the owner [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/21

Summary

`CommissionEscrow.bind()` calls `getAgentWallet()` unconditionally, while `ReviewRegistry._isAgentCaller()` reads it only when the caller is not the NFT owner.

Vulnerability Detail

`bind()` fetches both `ownerOf()` and `getAgentWallet()` before comparing them with `msg.sender`. When the caller is the owner, the second external call is wasted. `ReviewRegistry._isAgentCaller()` implements the same check but returns early for the owner.

Impact

Owner callers pay for an unnecessary external call, and the same check is written two different ways across contracts.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/CommissionEscrow.sol#L171-L175

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/ReviewRegistry.sol#L270-L275

Tool Used

Manual Review

Recommendation

Consider reading `agentWallet` only when the caller is not the owner:

```
-        address agentOwner = IERC721(_AGENT_REGISTRY).ownerOf(agentId);
-        address agentWallet =
↪   IIdentityRegistry(_AGENT_REGISTRY).getAgentWallet(agentId);
```

```
-        if (msg.sender != agentOwner && !(agentWallet != address(0) && msg.sender
↪  == agentWallet)) {
-            revert NotAgentController(agentId, msg.sender);
+        if (msg.sender != IERC721(_AGENT_REGISTRY).ownerOf(agentId)) {
+            address agentWallet =
↪  IIdentityRegistry(_AGENT_REGISTRY).getAgentWallet(agentId);
+            if (agentWallet == address(0) || msg.sender != agentWallet) revert
↪  NotAgentController(agentId, msg.sender);
         }
```

Issue L-10: Fallback grace period is a magic number [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/22

Summary

`triggerSettleFallback()` hardcodes the `10 days` grace period instead of using a named constant.

Vulnerability Detail

The grace period that decides when the permissionless fallback opens is an inline literal. It is not exposed or documented next to the other constants like `MIN_SETTLEMENT_WINDOW`.

Impact

The value is harder to find, read and change consistently.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/CommissionEscrow.sol#L290

Tool Used

Manual Review

Recommendation

Consider replacing the literal with a named constant:

```
+    uint256 public constant FALLBACK_GRACE_PERIOD = 10 days;
-        uint256 eligibleAt = uint256(rec.windowEnd) + 10 days;
+        uint256 eligibleAt = uint256(rec.windowEnd) + FALLBACK_GRACE_PERIOD;
```

Issue L-11: Purchases without a referral revert when the product has a commission [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/23

Summary

`_settleAndDeposit()` reverts with `MissingReviewerForCommission` when `reviewId == 0` and the product has any commission, so a commissioned product cannot be bought without a ref link.

Vulnerability Detail

`reviewId == 0` is documented as "no referral". `ReviewRegistry` describes it as the sentinel meaning "this purchase carries no review, pay no commission", and the README says commission is paid when a buyer purchases *through* a ref link. The router instead reverts the whole purchase, because `reviewerAgentId` defaults to `0`, which is a real ERC-8004 identity.

This has several side effects:

1. **Cold start.** A product with commission and no reviews cannot be bought, so no buyer can get the escrow record required by `publishReviewWithEscrowProof()` to write the first review.
2. **Forced referral.** A buyer who does not want to reward any existing reviewer (e.g. all reviews are negative) cannot buy without paying one of them.
3. **Misattribution.** To complete an organic sale, the facilitator or oracle must attach some unrelated `reviewId`, paying commission to a reviewer who referred nothing.
4. **Parked funds.** On `splitCharge()` the buyer has already paid, so a no-referral charge reverts after delivery and its USDC stays in the router until `recoverUSDC()`.
5. **Lost sale.** The merchant loses the sale and the protocol loses its platform fee.

The code also stores `reviewerAgentId = 0` on every no-referral escrow record (zero-commission or dust sales), which attributes the record to the real agent `0`. The README explicitly says agent `0` must never be treated as a sentinel.

Impact

Commissioned products cannot be bought or reviewed without a referral, which blocks organic sales and the first verified review.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L289-L294

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L304-L305

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/ReviewRegistry.sol#L32-L33

Tool Used

Manual Review

Recommendation

Consider settling no-referral purchases with zero commission, so the commission stays with the merchant, and recording a dedicated reviewer sentinel instead of agent `0`:

```
    function _computeSplitFor(uint256 amount, Attribution memory a) private view
    ↪  returns (SplitMath.Split memory s) {
        uint16 commissionBps =
            IProductRegistry(_PROTOCOL.getAddress(_PROTOCOL.PRODUCT_REGISTRY()))
                .getProduct(a.merchantId, a.productId)
                .commissionBps;
+       if (a.reviewId == 0) commissionBps = 0; // no referral: no commission
```

```
-       uint256 reviewerAgentId;
+       uint256 reviewerAgentId = ProtocolConstants.SENTINEL_REVIEWER; //
↪  type(uint256).max
```

With a sentinel reviewer, `CommissionEscrow.settleHeldBack()` should resolve the reviewer wallet only when `reviewerAmt > 0`, as the other escrow payout paths already do.

Issue L-12: Settlement reads the full product struct, including an unbounded URI, to get one rate [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/issues/24

Summary

`_computeSplitFor()` calls `getProduct()` to read only `commissionBps`, but the call copies the whole `Product` struct, including the unbounded `productCardURI` string.

Vulnerability Detail

`productCardURI` has no length limit and is set by the merchant or a `PRODUCT_MANAGER_ROLE` holder. Every `route()` and `splitCharge()` loads it from storage, ABI-encodes it and decodes it in the router, paying roughly one storage read per 32 bytes. A long URI is paid for once on write, but read again on every purchase of the product.

Impact

Every settlement of a product with a long URI costs the facilitator or submitter unnecessary gas.

Code Snippet

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/SplitRouter.sol#L502-L505

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/ProductRegistry.sol#L129-L132

https://github.com/sherlock-audit2/2026-09-opensouk-sep-21st-2026/blob/ace9677cc6292792ad26fff99ed794c9aa6b0b12/contracts-audit/src/interfaces/IProductRegistry.sol#L51

Tool Used

Manual Review

Recommendation

Consider adding a dedicated getter for the value the payment path needs, and bounding the URI length:

```
+    function getCommissionBps(uint256 merchantId, bytes32 productId) external view
 ↪  returns (uint16) {
+        _requireProduct(merchantId, productId);
+        return _products[merchantId][productId].commissionBps;
+    }
```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.